InterSystems
Creative data technology

Using Agentic AI with FHIR (Part 2)

Patrick W. Jamieson, M.D.
Technical Product Manager

Russ Leftwich, M.D.
Senior Clinical Advisor, Interoperability

Fan Ji,
Solution Developer

Agenda

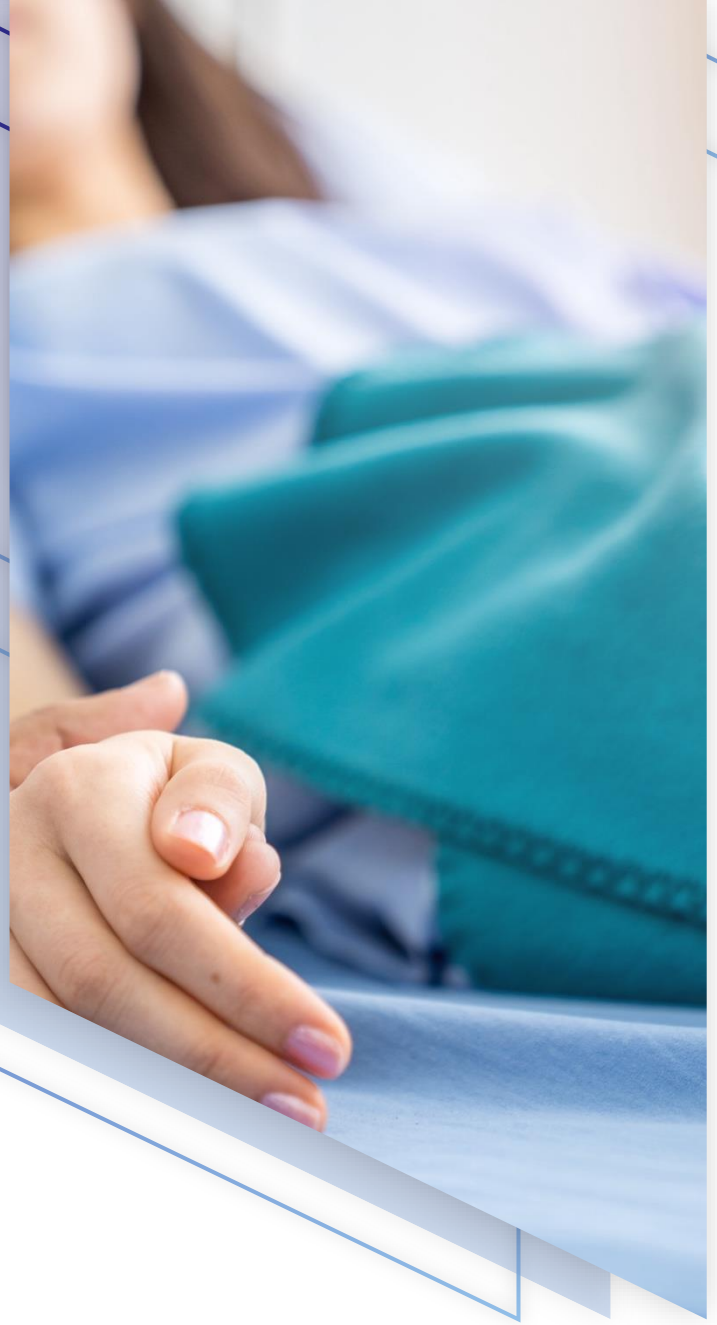
06 Patient retrieval with FHIR tools

07 From Bundle to compact clinical summary

08 Design patterns, safety, and production concerns

09 Q&A / Discussion

10 Want to Learn More



06: Patient Retrieval with FHIR Tools

Tools the Agent Can Use with the FHIR Repository



- FHIR_Read – you know the FHIR id of the patient you want information
- FHIR_Search – you want to look for specific patients based on FHIR search criteria for patients
- FHIR_Search – I want to find a list of Patients with a specific condition, requires terminology resolution with SNOMED CT, and reverse chaining with _has
- FHIR_Update – Update a FHIR resource, making a value substitution of a particular element , like a FHIR Patch operation. You must know the old value, the new value, and ”approve” the substitution.
- FHIR_everything – get all the resources in the patient compartment using the $everything operation (must know the FHIR id of the patient)
- FHIR_summarizer – (uses a subagentdefinition) – get all the resources in the patient compartment using $everything, use a bundle extractor, and create a formatted summary

FHIR_Read (Direct Retrieval)

Let’s see a demo of how this works using an Agent and a tool call FHIR_Read

Input: Patient ID = 123

GET /Patient/123

Returns single Patient resource

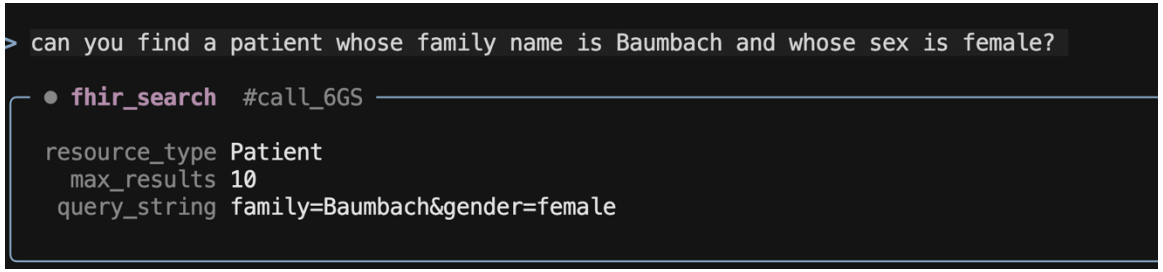
FHIR_Search (Basic Query)

Let’s see a demo of how this works using an Agent and a tool call FHIR_Search

Find female patients named Smith

GET /Patient?name=Smith&gender=female

Returns Bundle



FHIR_Search + Terminology Resolution

User Prompt : 'patients with diabetes'

- Resolve → SNOMED: 44054006 (this code is only for type 2 diabetes, what happens when we don’t specify the type of diabetes?

Under the hood, we need

- Token: system|code
- Reverse chaining

What is a Token Search in FHIR?

Definition

- A **token search** is used to match **coded values**
- Format: **system|code**

Why it matters

- Clinical data is stored as **standardized codes**, not free text
- Ensures **precise and interoperable queries**

Structure

- **system** → identifies the vocabulary (e.g., http://snomed.info/sct)
- **code** → the specific concept (e.g., 44054006)

Example

Find observations for glucose:

GET /Observation?code=http://loinc.org|2345-7

Reverse Chaining

Example

GET [base]/Patient?_has:Condition:patient:code=http://snomed.info/sct|44054006

All patients who have Type II Diabetes Mellitus

- Condition references Patient
- Need inverse traversal

Syntax
- _has:Condition:patient:code
- _has:Observation:subject:code
- Pattern: _has:<Resource>:<reference>:<field>

Agent Reasoning Flow

1

Extract concept

2

Resolve via MCP

3

Build _has query

4

Execute search

Demo Using Reverse Chaining

Find Patients with Conditions

Find Patients with Allergies



Patient Retrieval with FHIR Tools

Using the $everything Operation

How agents retrieve comprehensive patient data:

- GET /Patient/[id]/$everything returns complete longitudinal record
- Bundle may contain hundreds or thousands of resources
- Agent filters by resource type, date range, and clinical relevance
- Pagination handled automatically for large result sets

The $everything operation gives agents a comprehensive starting point for any clinical question.

FHIR Update

Agents can do more than read, they can write to the FHIR repository as well

- FHIR_update: replace an element string (similar to Patch)
- Require the user know the old and new value, limit to a single patient
- Get the approval of the user

Let’s see a demo

Why Approval is Required for FHIR Write Operations

The Problem

- FHIR write operations modify **clinical data**
- Actions may have **patient safety, legal, and audit implications**



Why Approval is Required for FHIR Write Operations

Examples of Risky Operations

- Updating a medication dosage
- Adding or modifying a diagnosis
- Writing allergy information
- Deleting or overwriting existing data

Why Agents Need Approval

- LLMs can **hallucinate or misinterpret intent**
- Clinical systems require **human oversight**
- Regulatory environments demand **traceability**

Key Insight

❌ Autonomous writes = unsafe

✅ Controlled writes with approval = production-ready AI

What is an Approval Policy?

An **Approval Policy** defines **when and how** an agent must seek approval before executing an action

Policy Components

- **Trigger Conditions**
  - Resource type (e.g., MedicationRequest, Condition)
  - Operation type (create, update, delete)
- **Approval Requirement**
  - Required vs optional
- **Approver Role**
  - Clinician, admin, pharmacist, etc.
- **Audit Requirements**
  - Log who approved, what changed, when

Example Policy

“Any update to MedicationRequest requires clinician approval”

“All Condition writes must be reviewed before commit”

Approval Flow in a FHIR Agent

End-to-End Flow

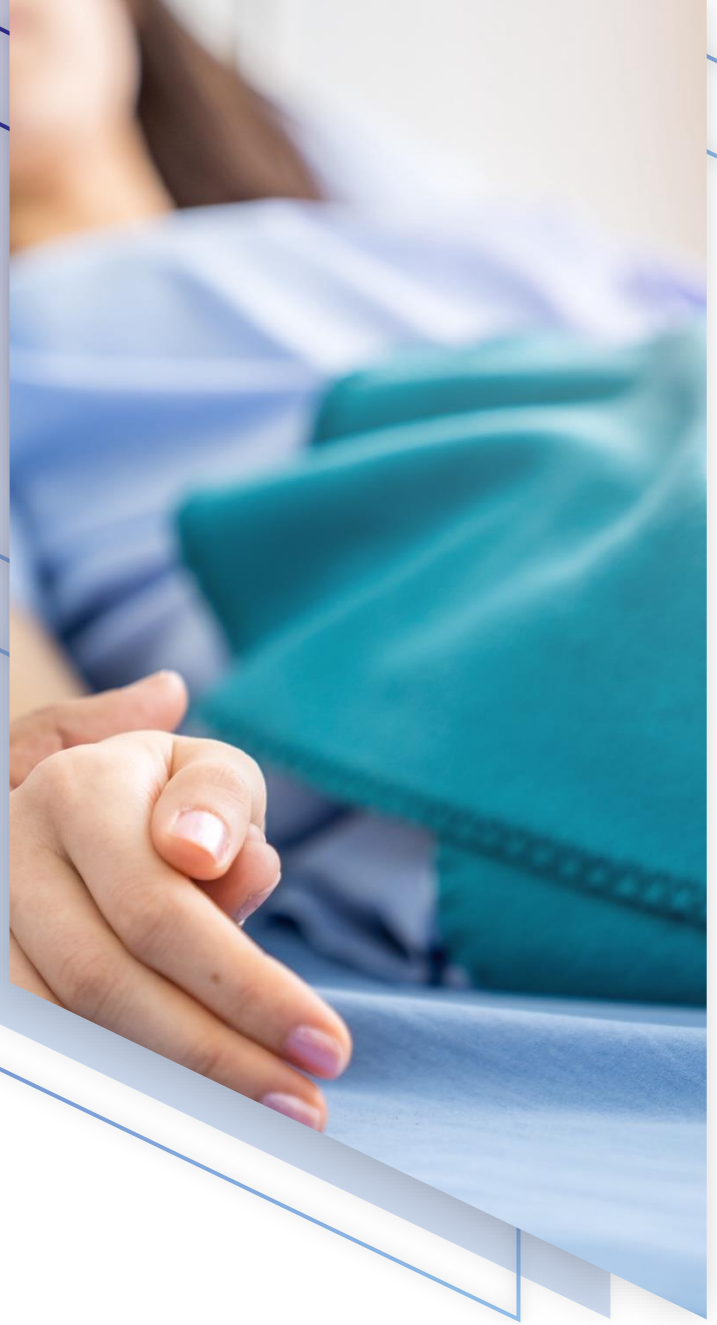
1. Agent determines write action
2. Policy engine evaluates request
3. Approval required → send for review
4. Human approves/rejects
5. Agent executes or aborts operation

Example

Agent: “Update medication dosage to 100mg”
Policy: “Medication updates require approval”
→ Approval request sent
→ Clinician approves
→ Agent performs FHIR update

Design Considerations

Synchronous vs asynchronous approval
UI vs API-based approval workflows
Timeout / escalation handling



07: From Bundle to Compact Summary

From Retrieval → Summarization



Bundle → filtering



Extract clinical signals



Produce summary

Creating an Effective Subagent Prompt

FHIR_SUMMARIZER = SubagentDefinition(
name="fhir_summarizer",

description="Automatically retrieves comprehensive patient data from a FHIR server, including demographics, clinical conditions, medications, allergies, and encounters, and generates a clear, concise, and clinically relevant patient summary report.",

goal_prompt="""
You are a clinical data summarization specialist working with FHIR healthcare data. If a user asks for a patient summary

and the patient id is provided, you will retrieve a patient's complete clinical record using the FHIR $everything operation.

The result will be a FHIR Bundle containing many resources.

Your job is to analyze the bundle and produce a concise clinical summary. If the user does not provide a patient id, you will return an error.

Creating an Effective Subagent Prompt

STEP 1 — Understand the Bundle

Identify the important resource types in the bundle.

Focus on:

Patient
Condition
Observation
MedicationRequest
Encounter
Procedure
AllergyIntolerance
DiagnosticReport
Immunization

Ignore administrative resources unless clinically relevant.

Creating an Effective Subagent Prompt



STEP 2 — Extract Key Clinical Facts

Extract the most important patient information:

Demographics
- name
- gender
- birthDate

Active Conditions
- diagnosis name
- status if available

Medications
- medication name
- status

Recent Encounters
- date
- encounter type

Recent Observations
- lab tests
- vital signs
- abnormal results

Allergies

Creating an Effective Subagent Prompt



```
----------------------------------------
STEP 3 — Produce a Patient Summary
----------------------------------------

Generate a concise structured summary in this format:

PATIENT SUMMARY

Name:
DOB:
Gender:

ACTIVE CONDITIONS
- ...

MEDICATIONS
- ...

RECENT ENCOUNTERS
- ...

RECENT OBSERVATIONS
- ...

ALLERGIES
- ...

NOTABLE CLINICAL FINDINGS
- ...

------------------------------
```

Creating an Effective Subagent Prompt

IMPORTANT RULES

Only summarize information present in the bundle.
Do NOT invent information.
If data is missing, say "Not available".

Be concise and clinically relevant.
""",
allowed_tools=["fhir_everything", "fhir_bundle_extract"],
max_turns=8
)

Production Concerns

Scaling Agentic FHIR Applications

Key considerations for production deployment:

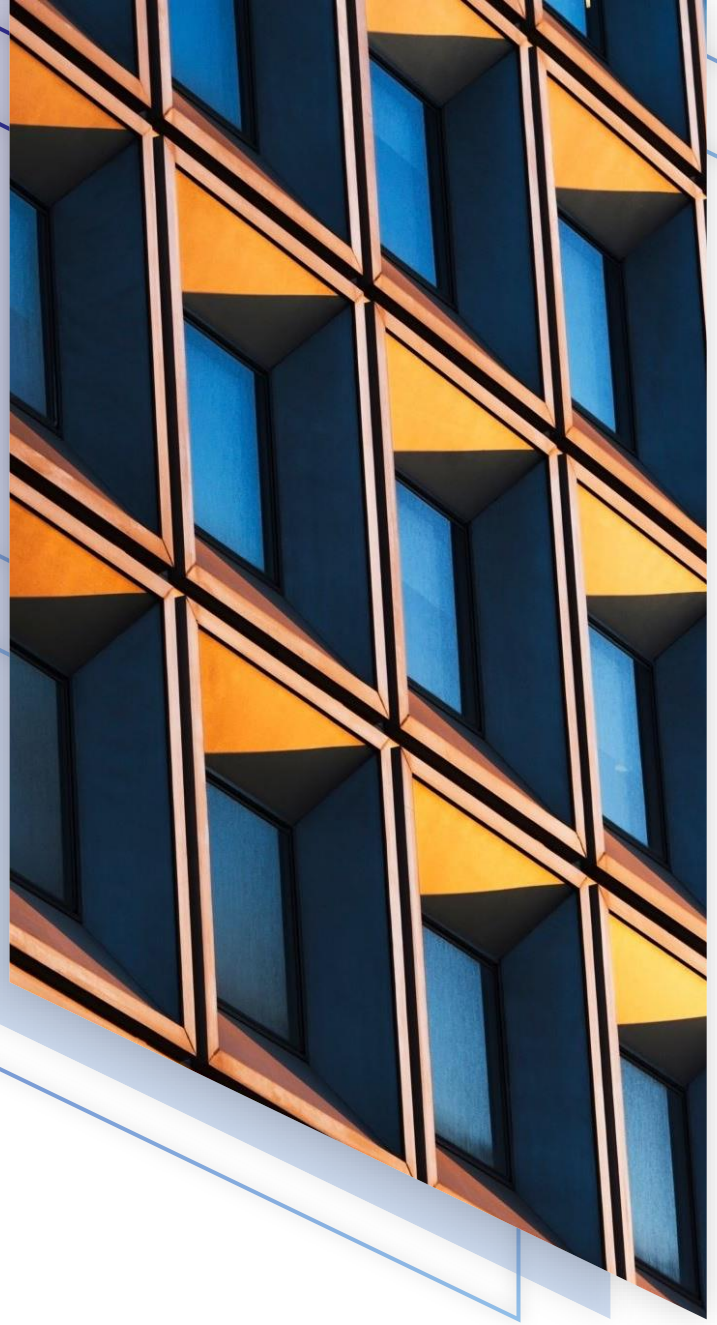
- Latency: minimize round-trips with batched FHIR queries
- Token management: compress clinical data before LLM calls
- Cost control: cache frequent terminology lookups and embeddings
- Compliance: ensure HIPAA-compliant data handling at every layer
- Monitoring: track agent decisions, tool calls, and error rates

Production readiness means treating **observability, cost, and compliance as first-class concerns**.

Demo Using $everything

Summarize Patient/123





08: Design patterns, safety, and production concerns

Design Pattern: Tool-Oriented Architecture

Agents don’t “do everything” — they orchestrate tools

Pattern

- LLM = reasoning layer
- Tools = execution layer
- Clear separation of concerns

Examples

- fhir_search → retrieval
- fhir_everything → bulk data
- terminology MCP → normalization
- subagents → specialized workflows

Key Insight

Better tools → simpler prompts → more reliable systems

Design Pattern: Subagents for Decomposition

Single-agent systems become brittle with complex tasks

Solution

- Break workflows into **specialized subagents**

Example

- Main Agent → delegates to:
  - FHIR Summarizer
  - Code Resolver
  - Cohort Builder

Benefits

- Scoped reasoning
- Reduced hallucination
- Reusable components

Key Insight

Subagents = **composable intelligence units**

Safety Pattern: Least Privilege Access

Default Principle - Agents should start with **read-only access**

Escalation Model

- Read → Analyze → Recommend
- Write → requires approval

Examples

- Read: Patient, Observation → safe
- Write: MedicationRequest → controlled
- Delete → highly restricted

Limit what the agent *can do*

Safety Pattern: Structured Validation as Guardrail

LLM outputs can be **syntactically valid but clinically wrong**

Solution

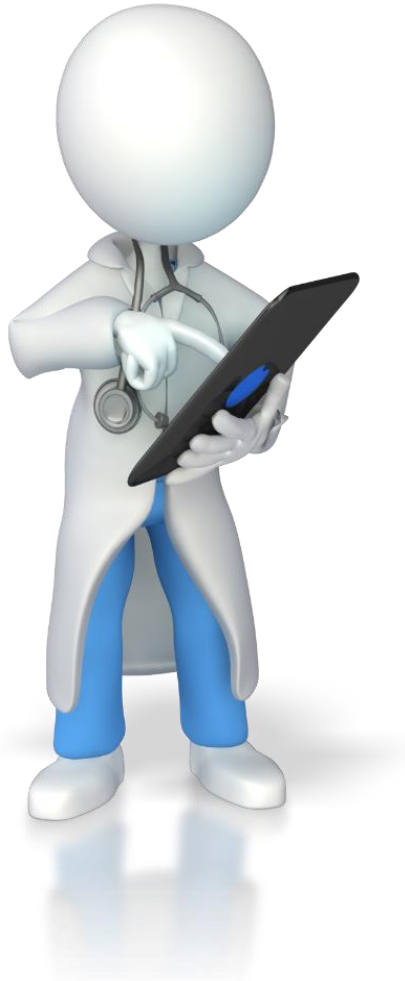
- Enforce **FHIR schema validation**
- Validate:
  - resourceType
  - required fields
  - code systems

What to Log

- MCP resolutions
- FHIR queries
- Selected codes
- Approval decisions

Why it Matters

- Debugging agent behavior
- Clinical audit trails
- Trust and transparency



Production Reality: Failure is the Norm



Common Failures

- Terminology resolution ambiguity
- FHIR server timeouts
- Empty search results
- Partial or inconsistent data

Design for Failure

- Retry strategies
- Fallback queries
- Graceful degradation
- User clarification loops



10: Want to Learn More

InterSystems AI Hub

join the EAP today!

Build native Agents on IRIS



Use a rich, intuitive SDK to build safe Agents, close to your data

Connect external Agents to IRIS using MCP



Expose business logic and data from IRIS safely using OAuth and MCP

Operationalize AI with IRIS



Add governance to your LangChain apps with IRIS security and auditing

Build safe AI apps & Agentic Workflows faster with the AI Hub, whether you're building on IRIS, wiring up existing IRIS investments, or looking for AI governance for external AI frameworks



Thank You